

Super Store Portal

Remaining Work — Modular Implementation Prompts for the Coding Agent

September 3, 2026

Contents

How To Use This Document	3
Current State (for your reference — not part of any prompt)	3
MODULE 0 — Critical Fix: Atomic Sale Completion (do this first)	3
Context	3
Scope	3
What to build	4
What NOT to do	4
Verification	4
MODULE 1 — Purchase Orders + Supplier Payments	4
Context	4
Scope	4
What to build	5
What NOT to do	5
Verification	6
MODULE 2 — Stripe Webhook Handler	6
Context	6
Scope	6
What to build	6
What NOT to do	6
Verification	7
MODULE 3 — User Management CRUD	7
Context	7
Scope	7
What to build	7
What NOT to do	7
Verification	8
MODULE 4 — Expense & Income Module (backend + frontend)	8
Context	8
Scope	8
What to build	8
What NOT to do	9
Verification	9

MODULE 5 — Reports + Date-Range Filtering	9
Context	9
Scope	9
What to build	9
What NOT to do	10
Verification	10
MODULE 6 — Sales Charts (optional, depends on Module 5)	10
Context	10
Scope	10
What to build	10
What NOT to do	10
Verification	11
MODULE 7 — Hardening: Validation, Rate Limiting, API Docs	11
Context	11
Scope	11
What to build	11
What NOT to do	11
Verification	11
Explicitly Out of Scope — Do Not Build	11

How To Use This Document

Each module below is a **complete, standalone prompt**. Copy **one module at a time** into the coding agent — don't paste the whole document at once. Finish, test, and commit one module before starting the next.

Why split it this way: if a session hits its context/usage limit mid-module, everything built in prior modules is already working and testable independently. You never lose progress on a module that's already shipped, and the next module's prompt carries enough restated context to start cold in a brand-new session — it doesn't depend on the agent remembering earlier conversations.

Order matters for the first three (Transaction Safety → Purchase Orders → Stripe Webhook) — everything after that can be done in any order you like, based on what you need next.

Current State (for your reference — not part of any prompt)

Built and working: Auth (Admin/Cashier JWT), RBAC, Products/Categories CRUD with Cloudinary images, Inventory (stock, adjustments, movement history), full POS (search, cart, customer select, Stripe payment), Self-Checkout Cart pickup/finalize (Pending Carts), Suppliers CRUD with traceability stats, Dashboard (Admin + Cashier), Receipts.

Models already in place (12): User, Customer, Product, Category, Supplier, Sale, Cart, Payment, Purchase, StockMovement, Income, Expense.

Note: Purchase and Income/Expense schemas already exist in the codebase, but their controllers/routes/frontend pages are incomplete or missing — the modules below build the missing layers on top of existing schemas where noted, not from scratch.

MODULE 0 — Critical Fix: Atomic Sale Completion (do this first)

Context

This app is a Super Store ERP + POS system (React 19, Express 5, MongoDB/Mongoose, Stripe, Cloudinary). Sale completion currently touches multiple collections (Sale, Payment, StockMovement, Income, and a stock decrement on Product) **without a real MongoDB transaction** — the only existing safety net is a manual cleanup path that deletes the Sale and cancels the Cart if Stripe PaymentIntent creation fails. That doesn't cover the actual completion step (payment succeeds → stock/sale/payment/income all commit together).

Scope

Wrap the entire sale-completion sequence in a single `mongoose.startSession()` transaction, used by **both** the POS flow and the self-checkout finalize flow — write it once as a shared service function, call it from both places.

What to build

1. A shared service function, e.g. `saleService.completeSale(saleId, session)`, that within one transaction:
 - For each item: `Product.findOneAndUpdate({ _id: productId, stockQuantity: { $gte: quantity } }, { $inc: { stockQuantity: -quantity } }, { session })`. If **any** item returns null (insufficient stock), abort the transaction — do not commit anything, and surface a clear error to the caller.
 - Create one `StockMovement` (type: "SALE") per item, capturing previous-Stock/newStock, within the same session.
 - Set `Sale.status = "completed"`.
 - Set `Payment.status = "succeeded"`, `paidAt = now`.
 - Create the Income record (referenceType: "sale", referenceId: `Sale._id`).
2. Call this function from wherever the POS confirms a Stripe payment succeeded, and from wherever the self-checkout payment confirmation happens — **replace any duplicated version of this logic in either place** with a call to the shared function.
3. Keep the existing "PaymentIntent creation failed → delete Sale, cancel Cart" cleanup as-is; that's a different failure case (before payment even starts) and doesn't need a transaction.

What NOT to do

- Do not touch Products/Categories/Inventory/POS UI — this is a backend correctness fix only, no new endpoints, no frontend changes.
- Do not change the Stripe integration itself, only what happens in MongoDB once Stripe confirms success.
- Do not add stock **reservation** (no cart-expiry-based holds) — the atomic conditional decrement above is the entire safety mechanism, exactly as already designed.

Verification

1. Manually reduce a product's stock to 1 in one browser tab; in another, attempt to buy 2 of it — confirm the sale is rejected before any partial write happens (no orphaned `StockMovement`, no `Sale` stuck in a weird state).
2. Simulate two near-simultaneous checkouts for the last unit of a product (e.g. two quick successive requests) — confirm only one succeeds.
3. Confirm a completed sale still produces exactly one `StockMovement` per item, one Income record, and the `Payment/Sale` status update, all in one commit.

MODULE 1 — Purchase Orders + Supplier Payments

Context

Same app as above. The Purchase schema already exists (`supplierId`, `items`, `totalAmount`, `status`, `paymentStatus`) but the workflow to actually create one — and have it affect stock/pricing/supplier balance — isn't built. Supplier currently has **no balance fields at all**, and there is no `SupplierPayment` model yet.

Scope

Complete the Purchase Order workflow, and add Supplier Payments as an extension of the existing Suppliers module.

What to build

1. Supplier schema update

Add to the existing Supplier model: openingBalance (Number, default 0), currentBalance (Number, default 0).

2. New model: SupplierPayment

```
SupplierPayment
├── supplierId
├── amount
├── date
├── method
├── reference
├── createdBy
└── timestamps
```

3. Backend — Purchases

```
POST /api/purchases { supplierId, items: [{ productId, quantity, purchasePrice }] }
GET  /api/purchases
GET  /api/purchases/:id
```

POST — in a single transaction (use the same `mongoose.startSession()` pattern from Module 0):

- Create the Purchase record (compute `totalAmount` server-side from the line items — never trust a frontend-supplied total).
- Increase each `Product.stockQuantity` by `quantity`.
- Overwrite each `Product.purchasePrice` with this purchase's `purchasePrice` (Last Purchase Price method) — **never touch `Sale.items[].purchaseCost` on any past sale**, those stay frozen as historical snapshots.
- Write one `StockMovement` (type: "PURCHASE") per line item.
- Increase `Supplier.currentBalance` by `totalAmount`.

4. Backend — Supplier Payments

```
POST /api/suppliers/:id/payments { amount, date, method, reference }
GET  /api/suppliers/:id/payments
```

POST creates the `SupplierPayment` and decreases `Supplier.currentBalance` by `amount`, in the same request.

5. Frontend

- New `Purchases.jsx` page + `purchaseService.js`: supplier picker, add product lines (product + quantity + purchase price), running total for review, **Save Purchase** submits as one request. List view + read-only detail drill-down.
- Extend the **existing** Supplier detail view (don't make a new page) with a **Record Payment** button (small form: amount, date, method, reference) and a payment history list below the existing purchase history.

What NOT to do

- Admin only — Cashier gets zero access to any of this, not even read-only.
- Do not make `currentBalance` directly editable on the Supplier form — it only ever moves via a recorded Purchase or SupplierPayment.
- Do not touch Sale documents or their `purchaseCost` snapshots.

Verification

1. Record a Purchase → confirm stock increases, purchasePrice updates, one StockMovement per line, currentBalance increases by the total.
 2. Record a Supplier Payment → confirm currentBalance decreases correctly.
 3. Attempt to delete a Supplier with purchase/payment history — confirm it's still blocked (existing deletion-protection rule) or handled sensibly.
 4. Confirm an old Sale's purchaseCost is unchanged after a new Purchase changes that product's price.
-

MODULE 2 — Stripe Webhook Handler

Context

Same app. Payments are currently confirmed via the client-side flow only. A webhook handler adds a server-authoritative fallback so payment status is never solely dependent on the frontend completing its callback.

Scope

Add a POST /api/webhooks/stripe endpoint that verifies Stripe's signature and reconciles Payment/Sale status independent of the client.

What to build

1. Webhook route registered **before** any body-parsing JSON middleware that would break signature verification — Stripe requires the **raw** request body for signature checks. Use `express.raw({ type: 'application/json' })` scoped to just this route.
2. Verify the signature using `STRIPE_WEBHOOK_SECRET` and the `stripe-signature` header — reject with 400 on failure, never process an unverified payload.
3. Handle at minimum: `payment_intent.succeeded` and `payment_intent.payment_failed`.
 - On `succeeded`: look up the Payment by `stripePaymentIntentId`; if not already status: "succeeded", call the **same** `saleService.completeSale()` from Module 0 (idempotent — if it's already completed, no-op).
 - On `payment_failed`: mark `Payment.status = "failed"`, leave Sale as pending/failed, don't touch inventory.
4. Store `webhookEventId` (already a field per your schema) on the Payment and check it before processing, to guard against Stripe's at-least-once delivery causing double-processing.

What NOT to do

- Do not skip signature verification "for now" — this endpoint is a security-sensitive entry point.
- Do not duplicate the sale-completion logic here — call the Module 0 shared service function.
- Do not require this endpoint to be authenticated with your app's JWT — Stripe calls it directly; security comes from signature verification, not your auth middleware.

Verification

1. Use the Stripe CLI (`stripe listen --forward-to localhost:5000/api/webhooks/stripe`) to trigger test events locally.
 2. Confirm a `payment_intent.succeeded` event correctly completes a Sale that hasn't been completed yet by the client-side flow.
 3. Confirm sending the same event twice doesn't create duplicate StockMovement/Income records (idempotency check works).
 4. Confirm a tampered/invalid signature is rejected with 400 and does nothing.
-

MODULE 3 — User Management CRUD

Context

Same app. Admin/Cashier accounts currently can only be seeded — there's no in-app way to create or manage them.

Scope

Admin-only CRUD for staff (User) accounts.

What to build

Backend

```
GET    /api/users
POST   /api/users      { name, email, password, role }
PUT    /api/users/:id
DELETE /api/users/:id
```

- Hash the password with the **same** bcrypt logic already used in `authController.js` — don't duplicate it, import/reuse it.
- DELETE/deactivate must reject (403) if `req.user.id === targetUserId` — an Admin can never remove their own account through this endpoint.

Frontend

New `Users.jsx` page + `userService.js`. Table: name, email, role, active status. Add/edit form. Deactivate/delete action hidden or disabled on the logged-in Admin's own row.

What NOT to do

- Admin only, no Cashier access whatsoever.
- No self-registration path for this role — Users page is the only way these accounts get created.
- Do not let this page touch the Customer collection — that's a completely separate model, unrelated to this task.

Verification

1. Create a new Cashier via this page, log out, log in as that account — confirm it works and lands on the Cashier dashboard.
 2. Attempt to deactivate/delete your own logged-in Admin account — confirm it's blocked.
 3. Attempt any of these endpoints as a Cashier via direct API call — confirm 403.
-

MODULE 4 — Expense & Income Module (backend + frontend)

Context

Same app. Both Expense and Income schemas already exist, but **no routes, controllers, or frontend pages exist yet for either** — this is a full module, not just a UI addition.

Scope

Full CRUD for Expenses (Admin), and Income with a critical rule: sale-generated income must stay read-only.

What to build

Backend — Expenses (plain CRUD)

```
GET    /api/expenses
POST   /api/expenses
PUT    /api/expenses/:id
DELETE /api/expenses/:id
```

category should be constrained to a fixed set (Electricity, Rent, Internet, Maintenance, Marketing, Transport, Other) — validate server-side, don't accept arbitrary strings.

Backend — Income

```
GET    /api/income
POST   /api/income      (manual entries only – referenceType: "manual")
PUT    /api/income/:id
DELETE /api/income/:id
```

Critical: PUT/DELETE must reject (403 or 409) any record where `referenceType !== "manual"`. Sale-linked income (created automatically by Module 0's `completeSale`) must never be editable or deletable through this API — it has to stay in lockstep with its Sale.

Frontend

- `Expenses.jsx` + `expenseService.js` — table with add/edit/delete, category filter, date-range filter.
- `Income.jsx` + `incomeService.js` — list shows both auto (sale-linked) and manual entries together; edit/delete controls appear **only** on manual rows. Sale-linked rows are read-only and link through to that Sale's detail instead.

What NOT to do

- Admin only for both.
- Do not allow editing the amount/referenceType/referenceId on any sale-linked Income record under any circumstance.
- Do not let this module write to Sale or Payment — it's a read-relationship only (Income references a Sale, never modifies it).

Verification

1. Add/edit/delete an Expense — confirm it persists and filters correctly.
 2. Complete a POS sale (Module 0's flow) — confirm the resulting Income record shows up here as read-only.
 3. Attempt PUT/DELETE on that sale-linked Income record via direct API call — confirm it's rejected.
 4. Add a manual Income entry — confirm it **is** editable/deletable.
-

MODULE 5 — Reports + Date-Range Filtering

Context

Same app. `/api/reports` currently only returns dashboard metrics (per your existing `reportController`) — there's no dedicated Reports page with per-domain breakdowns or date filtering.

Scope

Admin-only Reports page with live-aggregated, date-range-filterable reports across every domain.

What to build

Backend

```
GET /api/reports/sales?from=&to=
GET /api/reports/purchases?from=&to=
GET /api/reports/inventory
GET /api/reports/payments?from=&to=
GET /api/reports/expenses?from=&to=
GET /api/reports/income?from=&to=
GET /api/reports/profit-loss?from=&to=
```

Use MongoDB aggregation pipelines (`$match/$group/$sum`) for every one of these — do not pull all documents into Node and sum in application code.

Profit/Loss specifically:

```
Revenue    = SUM(Sale.total) where status = completed, within range
COGS       = SUM(Sale.items[].purchaseCost * quantity) across those same sales
Expenses   = SUM(Expense.amount) within range
Net Profit = Revenue - COGS - Expenses
```

COGS must read from each sale's historical purchaseCost snapshot — never from the current live `Product.purchasePrice`.

Frontend

New `Reports.jsx` page — a tab or dropdown per report type, each rendering a table. One shared date-range picker (`from/to`) applies to whichever report is currently open.

What NOT to do

- Admin only, no Cashier access.
- Do not cache any of these numbers — every request recomputes live from MongoDB.
- Do not build charts here — that's Module 6, kept separate on purpose so this module can ship and be tested without a charting library dependency.

Verification

1. Pull each report for a known date range, manually spot-check totals against the raw collections.
 2. Pull Profit/Loss and manually verify Revenue – COGS – Expenses matches, using historical `purchaseCost`.
 3. Confirm changing the date range actually changes every number (not just the Sales tab).
-

MODULE 6 — Sales Charts (optional, depends on Module 5)

Context

Same app, building on Module 5's Reports endpoints.

Scope

Add visual charts to the Sales report tab — this is a frontend-only addition, no new backend endpoints needed if Module 5's `/api/reports/sales` already returns date-wise breakdowns.

What to build

- Add `Recharts` or `Chart.js` (your call — `Recharts` tends to be simpler with `React`) as a new frontend dependency.
- A line or bar chart of sales-over-time on the Sales report tab, using the existing `/api/reports/sales?from=&to=` data.
- Optionally: a simple bar chart for Inventory (stock levels) or Expenses (by category) if time permits — treat these as stretch, not required.

What NOT to do

- Do not add a new backend endpoint just for chart data — reuse Module 5's existing aggregation response shape; reshape it client-side if needed.
- Skip this module entirely if you're tight on time/context — it's the lowest-value item in the "Medium" priority group and has zero dependents.

Verification

Chart renders correctly against real data and updates when the date range changes.

MODULE 7 — Hardening: Validation, Rate Limiting, API Docs

Context

Same app. Listed technical debt: no systematic input validation, no rate limiting, no API documentation.

Scope

Three independent, low-risk additions — do them in any order, or split across sessions individually if needed.

What to build

1. **Input validation** — add express-validator (or similar) to the highest-risk write endpoints first: /api/auth/login, /api/products (POST/PUT), /api/sales, /api/purchases, /api/customers. Return 422 with a clear field-level error on failure, matching the existing error response shape ({ success: false, message }).
2. **Rate limiting** — add express-rate-limit on /api/auth/login at minimum (prevent brute force), and consider a general API-wide limiter as a second, looser layer.
3. **API documentation** — add a Swagger/OpenAPI spec (swagger-jsdoc + swagger-ui-express is a common pairing) covering the routes that exist today. This can be built incrementally, module by module, rather than all at once.

What NOT to do

- Do not change any business logic while adding validation — this is purely a guard layer in front of existing behavior.
- Do not rate-limit webhook endpoints (Module 2) the same way as user-facing ones — Stripe's retry behavior needs headroom.

Verification

1. Submit malformed data to each validated endpoint — confirm clear 422 errors, not a raw stack trace or a 500.
 2. Attempt rapid repeated login attempts — confirm rate limiting kicks in.
 3. Open the generated Swagger UI and confirm it accurately reflects real request/response shapes.
-

Explicitly Out of Scope — Do Not Build

Item	Why
Barcode Scanner	Contradicts the finalized spec (§3): “Out of scope: cash drawer integration, barcode scanner, card reader, Stripe Terminal hardware.” Product search is by name/category text only, by design.
Multi-Store Support	Contradicts the project’s core premise: a single physical store. None of the existing schemas (Sale, Product, Inventory, etc.) have a storeId/branch concept — this would be a major architectural rework, not an incremental module.
Email Notifications	Low priority per your own list, and no SMTP configuration exists in this app’s environment (SMTP belongs to the separate store customer-facing app, for OTP registration only) — don’t add SMTP config here without an explicit decision to do so.
Receipt PDF Generation	Low priority, optional. Current receipts already render correctly for browser printing (<code>window.print()</code>); server-side PDF generation is a nice-to-have, not a gap. Build only if specifically requested later.

End of module set. Each module above is self-contained — start any fresh agent session with just that module’s text.